

Clustering Algorithms

1 Introduction to Clustering

Clustering is an **unsupervised learning** technique that aims to group a set of data objects into clusters such that objects within the same cluster are more similar to each other than to those in other clusters. Similarity is usually measured using distance metrics such as Euclidean, Manhattan, or cosine distance.

1.1 Objectives of Clustering

Clustering aims to organize unlabeled data into meaningful groups based on similarity or proximity measures. Since no prior class labels are available, clustering plays a fundamental role in exploratory data analysis and unsupervised learning. The key objectives are explained in detail below.

- **Discover hidden patterns in data:** Real-world datasets are often large, complex, and high-dimensional, making it difficult to identify underlying structures through manual inspection. Clustering helps uncover hidden patterns, trends, or regularities by grouping similar data points together. These patterns may correspond to unknown categories, behavioral trends, or functional similarities that were not previously apparent.
- **Reduce data complexity and improve interpretability:** By grouping a large number of data points into a smaller number of clusters, clustering significantly reduces data complexity. Each cluster can be represented using a prototype such as a centroid or medoid, allowing analysts to summarize and interpret massive datasets more efficiently. This abstraction is especially useful for visualization, reporting, and decision-making.
- **Identify natural groupings or intrinsic structure:** Many datasets possess inherent group structures, such as customer segments, document topics, or biological classes. Clustering algorithms aim to identify these natural groupings without requiring external supervision. This helps in understanding the internal organization of data and provides insights into the domain being analyzed.
- **Serve as a preprocessing step for other learning tasks:** Clustering is often used as a preprocessing or intermediate step in machine learning pipelines. It can help generate pseudo-labels, initialize parameters for supervised models, reduce training data size, or improve classification accuracy. Additionally, clustering-based representations are frequently used in anomaly and novelty detection, where data points far from any cluster are treated as outliers.

1.2 Applications

Clustering algorithms are widely used across diverse domains due to their ability to analyze unlabeled data and extract meaningful information. Some important applications are described below.

- **Data mining and knowledge discovery:** In data mining, clustering is used to explore large databases, discover hidden structures, and group similar records together. It forms a core component of knowledge discovery systems and is commonly applied in customer behavior analysis, web usage mining, and document clustering.
- **Image segmentation:** Clustering techniques such as K-means and Fuzzy C-means are extensively used in image processing to divide images into homogeneous regions based on color, intensity, or texture. Image segmentation is a crucial step in applications such as object recognition, medical image analysis, satellite imagery, and computer vision.
- **Bioinformatics and gene expression analysis:** In bioinformatics, clustering is applied to gene expression data, protein sequences, and biological networks. It helps identify co-expressed genes, functional gene groups, and disease subtypes. Such clustering-based analysis supports biological interpretation and hypothesis generation in genomics and systems biology.
- **Market segmentation and business analytics:** Organizations use clustering to divide customers into distinct segments based on purchasing behavior, demographics, or preferences. This enables targeted marketing, personalized recommendation systems, customer retention strategies, and improved business decision-making.
- **Social network analysis and community detection:** Clustering is widely used to detect communities within social networks by grouping users with strong interaction or similarity patterns. Community detection aids in understanding information diffusion, influence analysis, recommendation systems, and social behavior modeling.

1.3 Types of Clustering Algorithms

Clustering algorithms can be broadly classified based on how clusters are formed, represented, and interpreted. Each category makes different assumptions about data distribution, cluster shape, and membership. The major types of clustering algorithms are described below.

1. **Partition-based clustering:** Partition-based clustering divides a dataset into a predefined number of clusters such that each data point belongs to exactly one cluster, typically by optimizing an objective function like minimizing intra-cluster variance or distance. Clusters are represented using prototypes such as centroids or medoids, making these methods computationally efficient and suitable for large datasets with well-separated, compact clusters; however, they require prior specification of the number of clusters and are sensitive to initialization and outliers (e.g., K-means, K-medoids, PAM).

2. **Hierarchical clustering:** Hierarchical clustering builds a nested hierarchy of clusters represented by a dendrogram, either through agglomerative (bottom-up) merging or divisive (top-down) splitting of clusters. It does not require the number of clusters to be specified in advance and provides an interpretable structure based on linkage criteria such as single, complete, average, or Ward's method, but it is computationally expensive for large datasets and sensitive to noise.
3. **Density-based clustering:** Density-based clustering identifies clusters as dense regions of data points separated by areas of low density, allowing the discovery of arbitrarily shaped clusters while naturally detecting noise and outliers. These methods define clusters based on neighborhood density rather than distance to a centroid and do not require the number of clusters to be specified, though their performance depends heavily on parameter selection and struggles with datasets of varying density (e.g., DBSCAN, OPTICS).
4. **Fuzzy clustering:** Fuzzy clustering allows each data point to belong to multiple clusters with varying degrees of membership, reflecting uncertainty or overlap in cluster boundaries. Instead of hard assignments, membership values lie between 0 and 1, making fuzzy clustering particularly suitable for real-world data with gradual transitions between clusters, although it is computationally more expensive and sensitive to noise and parameter settings (e.g., Fuzzy C-Means).
5. **Model-based clustering:** Model-based clustering assumes that data is generated from a mixture of underlying probabilistic models, with each cluster represented by a probability distribution such as a Gaussian. Using statistical inference techniques like the Expectation–Maximization algorithm, these methods provide soft probabilistic cluster assignments and a strong theoretical foundation, enabling modeling of overlapping clusters and uncertainty, but they require distributional assumptions and are computationally intensive (e.g., Gaussian Mixture Models).

2 K-Means Clustering

2.1 Basic Idea

K-means is one of the most widely used partition-based clustering algorithms in unsupervised learning. It aims to divide a dataset consisting of n data points into k non-overlapping clusters, where each cluster is represented by its centroid, defined as the arithmetic mean of the data points belonging to that cluster. The algorithm iteratively refines cluster assignments so that data points within the same cluster are as similar as possible, while those in different clusters are as dissimilar as possible, typically measured using Euclidean distance.

2.2 Objective Function

The objective of the K-means algorithm is to minimize the *within-cluster sum of squared errors* (WCSS), also known as the clustering distortion or inertia. Mathematically, the

objective function is given by:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2 \quad (1)$$

where C_i denotes the set of data points assigned to the i -th cluster and μ_i represents the centroid of that cluster. Minimizing J ensures compact and well-separated clusters by reducing intra-cluster variance.

2.3 Algorithm Description

K-means operates through an iterative refinement process involving two main steps: assignment and update. The algorithm converges when cluster assignments no longer change or when the reduction in the objective function becomes negligible.

Algorithm 1 K-Means Algorithm

- 1: Select the number of clusters k
 - 2: Initialize k centroids randomly or using a heuristic
 - 3: **repeat**
 - 4: Assign each data point to the cluster with the nearest centroid
 - 5: Update each centroid as the mean of the data points assigned to it
 - 6: **until** no change in centroids or cluster assignments
-

2.4 Initialization Strategies

The performance of K-means is highly dependent on the initial choice of centroids. Poor initialization may lead to slow convergence or suboptimal local minima. Common initialization strategies include random selection of centroids, multiple random restarts, and the K-means++ method, which spreads initial centroids to improve clustering quality.

2.5 Worked Example

Consider a two-dimensional dataset with points $(2, 2)$, $(3, 3)$, $(8, 8)$, and $(9, 9)$, and let the number of clusters be $k = 2$. If the initial centroids are chosen near $(2, 2)$ and $(8, 8)$, the algorithm assigns the first two points to the first cluster and the remaining points to the second cluster. After updating centroids, the final cluster centers converge to approximately $(2.5, 2.5)$ and $(8.5, 8.5)$, yielding two compact and well-separated clusters.

2.6 Distance Measures

Although Euclidean distance is most commonly used in K-means, other distance measures such as Manhattan or cosine distance may be employed depending on the nature of the data. However, the use of non-Euclidean distances may affect the interpretation of the centroid as a mean.

2.7 Convergence and Complexity

K-means is guaranteed to converge in a finite number of iterations, as each iteration monotonically decreases the objective function. The time complexity of the algorithm is $O(nkdt)$, where n is the number of data points, d is the dimensionality of the data, and t is the number of iterations.

2.8 Advantages and Limitations

Advantages:

- Simple to understand and easy to implement.
- Computationally efficient and scalable to large datasets.
- Works well when clusters are spherical and well separated.

Limitations:

- Requires prior specification of the number of clusters k .
- Sensitive to initial centroid selection and may converge to local minima.
- Performs poorly in the presence of outliers and non-spherical clusters.
- Not suitable for categorical or mixed-type data without modification.
- Tends to capture only round-shaped similar-sized clusters only.

3 K-Medoids Clustering

3.1 Concept

K-medoids is a partition-based clustering algorithm closely related to K-means, with the key difference that each cluster is represented by a *medoid*, which is an actual data point belonging to the dataset. Unlike centroids in K-means, medoids are less affected by extreme values, making K-medoids significantly more robust to noise and outliers. This property allows K-medoids to perform well in applications where the data contains anomalies or where the mean is not a meaningful representative.

3.2 Cluster Representation

In K-medoids, a medoid is defined as the data point within a cluster whose average dissimilarity to all other points in the cluster is minimal. Since medoids are real observations, K-medoids can be applied to datasets where only pairwise distances are known and explicit coordinate representations are unavailable.

3.3 Objective Function

The objective of K-medoids is to minimize the total dissimilarity between data points and their respective cluster medoids. The objective function is expressed as:

$$J = \sum_{i=1}^k \sum_{x \in C_i} d(x, m_i) \quad (2)$$

where C_i denotes the set of points assigned to the i -th cluster, m_i is the medoid of that cluster, and $d(\cdot, \cdot)$ represents an arbitrary distance or dissimilarity measure.

3.4 Algorithm Overview

The K-medoids algorithm iteratively refines cluster assignments by alternating between selecting medoids and assigning points to the nearest medoid. Popular implementations, such as PAM (Partitioning Around Medoids), explicitly search for better medoids by evaluating potential swaps between current medoids and non-medoid points.

3.5 Distance Measures

A major strength of K-medoids is its flexibility in using different distance measures, including Euclidean, Manhattan, cosine, and general dissimilarity matrices. This makes the algorithm suitable for non-Euclidean spaces and categorical or mixed-type data.

3.6 Worked Example

Consider a dataset containing several outliers. While K-means may shift centroids toward extreme values, K-medoids selects representative data points as cluster centers, thereby preserving the true structure of the data and maintaining cluster stability.

3.7 Computational Complexity

K-medoids is computationally more expensive than K-means, particularly for large datasets. The time complexity depends on the implementation but is generally quadratic or higher in the number of data points, limiting its scalability.

3.8 Advantages and Disadvantages

Advantages:

- Robust to noise and outliers due to medoid-based representation
- Supports arbitrary distance or dissimilarity measures
- Suitable for datasets with categorical or mixed attributes

Disadvantages:

- Computationally expensive for large-scale datasets
- Not well-suited for very high-dimensional data
- Slower convergence compared to K-means

4 Partitioning Around Medoids (PAM)

4.1 Overview

Partitioning Around Medoids (PAM) is a classical and widely studied algorithm for solving the K-medoids clustering problem. Unlike K-means, which represents clusters using centroids that may not correspond to actual data points, PAM identifies representative objects called *medoids* from the dataset itself. The algorithm explicitly searches for an optimal set of medoids by minimizing the total dissimilarity between data points and their assigned medoids, making it robust to noise and outliers.

4.2 Intuition Behind PAM

The central idea of PAM is to iteratively improve an initial selection of medoids by exploring potential swaps between current medoids and non-medoid data points. A swap is accepted if it leads to a reduction in the overall clustering cost. By systematically evaluating such swaps, PAM attempts to approximate a global optimum of the K-medoids objective function.

4.3 Objective Function

Given a dataset of n objects and a dissimilarity measure $d(\cdot, \cdot)$, PAM aims to minimize the total cost:

$$J = \sum_{i=1}^k \sum_{x \in C_i} d(x, m_i) \quad (3)$$

where m_i is the medoid of cluster C_i . The objective function measures how well the selected medoids represent the dataset.

4.4 Algorithm Description

PAM consists of two main phases: *Build* and *Swap*.

Algorithm 2 Partitioning Around Medoids (PAM)

- 1: Select k initial medoids from the dataset (randomly or using a heuristic)
 - 2: **repeat**
 - 3: Assign each data point to the nearest medoid based on dissimilarity
 - 4: Compute the total clustering cost (J)
 - 5: For each medoid m and each non-medoid point x :
 - 6: Evaluate the cost of swapping m with x
 - 7: If the swap reduces the total cost, record the improvement
 - 8: Perform the swap that yields the maximum cost reduction
 - 9: **until** no swap leads to further improvement
-

4.5 Build Phase

In the build phase, PAM selects an initial set of k medoids. This can be done randomly or using greedy heuristics that choose objects minimizing the overall dissimilarity. A good initialization helps reduce convergence time and improves clustering quality.

4.6 Swap Phase

During the swap phase, PAM systematically examines all possible swaps between current medoids and non-medoid points. For each potential swap, the algorithm recalculates the change in clustering cost. If a swap leads to a decrease in the objective function, it is considered beneficial. The best swap is executed in each iteration, ensuring monotonic improvement of the solution.

4.7 Convergence Properties

PAM guarantees convergence in a finite number of iterations because each accepted swap strictly reduces the objective function and the number of possible medoid configurations is finite. However, like most clustering algorithms, PAM may converge to a local optimum depending on initialization.

4.8 Computational Complexity

The time complexity of PAM is $O(k(n - k)^2)$ per iteration, where n is the number of data points and k is the number of clusters. Due to this high computational cost, PAM is suitable only for small to medium-sized datasets. For large datasets, scalable variants such as CLARA and CLARANS are preferred.

4.9 Advantages and Limitations

Advantages:

- Robust to noise and outliers due to medoid-based representation
- Works with arbitrary distance or dissimilarity measures
- Produces more interpretable cluster representatives

Limitations:

- Computationally expensive and not scalable to large datasets
- Sensitive to initial medoid selection
- Slower convergence compared to centroid-based methods

4.10 Applications

PAM is particularly useful in domains where robustness and interpretability are more important than scalability, such as bioinformatics, document clustering, and small-scale market segmentation tasks.

5 Fuzzy C-Means (FCM)

5.1 Motivation

Unlike hard clustering methods, where each data point belongs to exactly one cluster, fuzzy clustering allows a data point to belong to multiple clusters with different degrees of membership. This is particularly useful when cluster boundaries are overlapping or not well defined, which is common in real-world data.

5.2 Basic Concept

In Fuzzy C-Means, each data point x_j is associated with a set of membership values $\{u_{ij}\}$, where $u_{ij} \in [0, 1]$ denotes the degree of membership of the j -th data point in the i -th cluster. The memberships satisfy the constraint:

$$\sum_{i=1}^c u_{ij} = 1, \quad \forall j = 1, 2, \dots, n$$

where c is the number of clusters and n is the number of data points.

5.3 Objective Function

The objective function of FCM is defined as:

$$J_m = \sum_{i=1}^c \sum_{j=1}^n u_{ij}^m \|x_j - c_i\|^2 \quad (4)$$

where c_i is the centroid of the i -th cluster, u_{ij} is the membership of data point x_j in cluster i , and $m > 1$ is the *fuzziness parameter* controlling the degree of cluster overlap.

5.4 Update Equations

The minimization of J_m under the membership constraints leads to the following update rules:

$$u_{ij} = \frac{1}{\sum_{k=1}^c \left(\frac{\|x_j - c_i\|}{\|x_j - c_k\|} \right)^{\frac{2}{m-1}}}, \quad (5)$$

$$c_i = \frac{\sum_{j=1}^n u_{ij}^m x_j}{\sum_{j=1}^n u_{ij}^m} \quad (6)$$

5.5 Algorithm Description

The FCM algorithm proceeds iteratively as follows:

1. Initialize the membership matrix $U = [u_{ij}]$ such that $\sum_{i=1}^c u_{ij} = 1$.
2. Compute cluster centers using the current membership values.
3. Update membership values based on the distance between data points and cluster centers.
4. Repeat the above steps until convergence.

5.6 Advantages and Limitations

Advantages:

- Effectively models overlapping clusters
- Provides soft cluster assignments with uncertainty information
- More expressive than hard clustering methods

Limitations:

- Sensitive to noise and outliers
- Requires prior specification of the number of clusters
- Computationally more expensive than K-means

5.7 Applications

- Image segmentation and pattern recognition
- Medical diagnosis and biomedical data analysis

6 Hierarchical Clustering

6.1 Overview

Hierarchical clustering is an unsupervised clustering technique that organizes data into a hierarchy of nested clusters. Unlike partition-based methods, hierarchical clustering does not require the number of clusters to be specified in advance. Instead, it produces a multilevel clustering structure that can be visualized using a tree-like representation known as a dendrogram.

6.2 Types of Hierarchical Clustering

Hierarchical clustering algorithms are broadly classified based on the direction in which the hierarchy is constructed.

- **Agglomerative (bottom-up) clustering:** Agglomerative clustering begins with each data point treated as an individual cluster. At each iteration, the two closest clusters are merged based on a predefined linkage criterion. This merging process continues until all data points are combined into a single cluster. Agglomerative clustering is the most commonly used form of hierarchical clustering due to its simplicity and efficiency.
- **Divisive (top-down) clustering:** Divisive clustering starts with all data points grouped into a single cluster and recursively splits clusters into smaller subclusters. The division process continues until each data point forms its own cluster or a stopping criterion is met. Although conceptually appealing, divisive methods are computationally more expensive and less frequently used in practice.

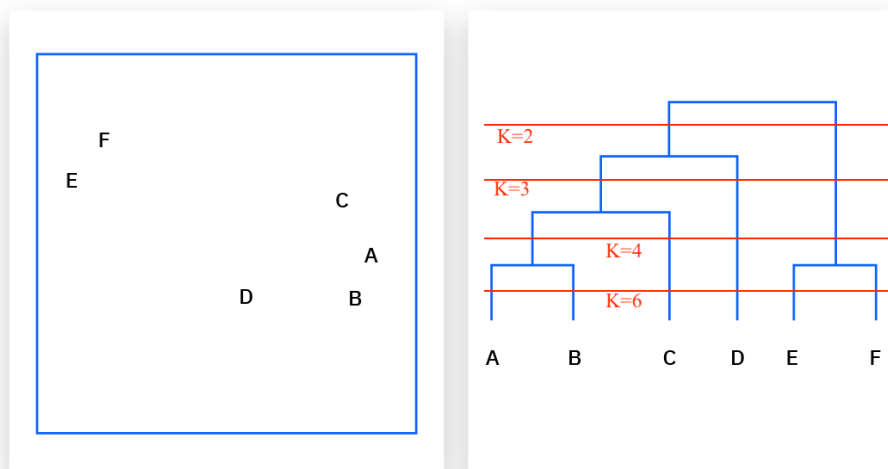
6.3 Linkage Criteria

The linkage criterion determines how the distance between two clusters is computed during the merging or splitting process. Different linkage methods lead to different clustering structures.

- **Single linkage:** Defines the distance between two clusters as the minimum distance between any pair of data points belonging to the two clusters. Single linkage is effective in detecting elongated or arbitrarily shaped clusters but is sensitive to noise and may suffer from the chaining effect.
- **Complete linkage:** Defines the distance between two clusters as the maximum distance between any pair of points from the two clusters. This method produces compact and well-separated clusters but may break large clusters and is sensitive to outliers.
- **Average linkage:** Computes the distance between two clusters as the average distance between all pairs of points across the clusters. Average linkage provides a compromise between single and complete linkage and generally yields balanced clustering results.
- **Ward's method:** Merges clusters in a way that minimizes the increase in total within-cluster variance after merging. Ward's method tends to create clusters of relatively equal size and is particularly effective when clusters are approximately spherical.

6.4 Dendrogram

A dendrogram is a graphical representation of the hierarchical clustering process, illustrating the order in which clusters are merged or split. The height at which two clusters are joined represents the dissimilarity between them. By cutting the dendrogram at a chosen height, a desired number of clusters can be obtained, making dendrograms a powerful tool for cluster interpretation and selection. The figure below shows example of dendrogram and how it is cut at different levels to obtain different number of clusters.



6.5 Algorithmic Characteristics

Hierarchical clustering relies on a distance matrix and repeatedly updates inter-cluster distances based on the chosen linkage criterion. Once a merge or split is performed, it cannot be undone, which makes hierarchical clustering deterministic but sensitive to early decisions.

6.6 Advantages and Limitations

Advantages:

- Does not require prior specification of the number of clusters
- Produces an interpretable hierarchical structure
- Applicable to a wide variety of distance measures

Limitations:

- Computationally expensive with time complexity $O(n^2)$ or higher
- Not suitable for very large datasets
- Sensitive to noise and outliers
- Lack of flexibility once clusters are merged or split

7 Density-Based Clustering: DBSCAN

7.1 Basic Idea

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together data points that are closely packed in high-density regions while identifying points in low-density regions as noise or outliers. Unlike partition-based methods, DBSCAN does not assume any specific cluster shape and is capable of discovering clusters of arbitrary geometry. It also does not require the number of clusters to be specified in advance, making it particularly suitable for exploratory data analysis.

7.2 Key Parameters

DBSCAN relies on two user-defined parameters that control the notion of density:

- **ϵ (Eps):** The radius of the neighborhood around a data point. It defines how close points must be to be considered neighbors.
- **MinPts:** The minimum number of points required within an ϵ -neighborhood (including the point itself) for a point to be considered dense.

The choice of these parameters significantly affects the resulting clusters and noise detection.

7.3 Density Concepts

Based on the parameters ε and MinPts, DBSCAN classifies data points into the following categories:

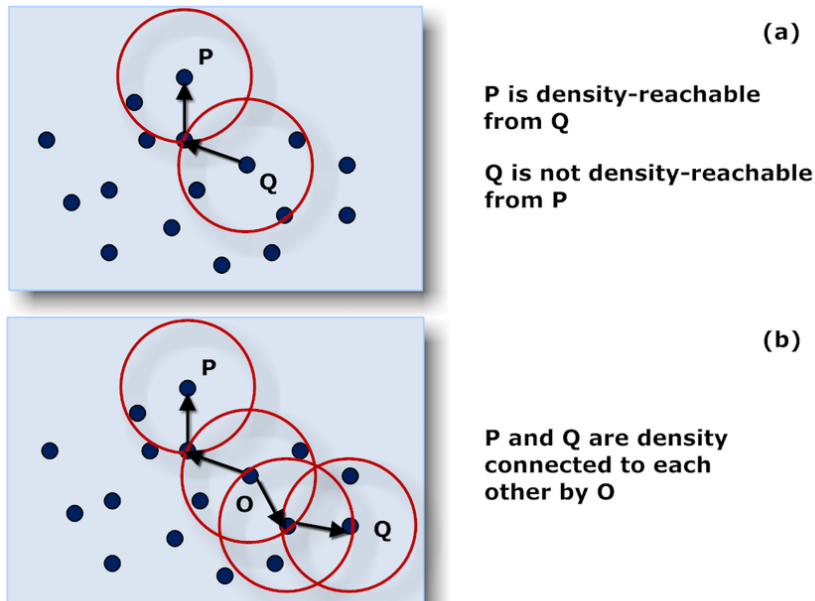
- **Core point:** A point is a core point if the number of points in its ε -neighborhood is at least MinPts. Core points lie in dense regions and form the backbone of clusters.
- **Border point:** A border point has fewer than MinPts points in its ε -neighborhood but lies within the ε -neighborhood of a core point. Border points are part of a cluster but do not themselves satisfy the density requirement.
- **Noise point (outlier):** A noise point is neither a core point nor a border point. Such points lie in sparse regions and are not assigned to any cluster.

7.4 Density Reachability and Connectivity

DBSCAN defines clusters using the notions of density reachability and density connectivity:

- A point p is *directly density-reachable* from a point q if p lies within the ε -neighborhood of q and q is a core point.
- A point p is *density-reachable* from q if there exists a chain of points $p_1, p_2, \dots, p_k$ such that $p_1 = q$ and $p_k = p$, where each point is directly density-reachable from the previous one.
- Two points are *density-connected* if there exists a point from which both are density-reachable.

See the figure below for illustration of reachability and connectedness. A cluster in DBSCAN is defined as a maximal set of density-connected points.



7.5 Algorithm Description

DBSCAN performs clustering by iteratively expanding dense regions starting from core points.

Algorithm 3 DBSCAN Algorithm

```
1: Mark all points as unvisited
2: for each unvisited point  $p$  do
3:   Mark  $p$  as visited
4:   Find the  $\varepsilon$ -neighborhood of  $p$ 
5:   if  $p$  is a core point then
6:     Create a new cluster containing  $p$ 
7:     Recursively add all density-reachable points to the cluster
8:   else
9:     Mark  $p$  as noise
10:  end if
11: end for
```

7.6 Worked Intuition

Starting from a core point, DBSCAN expands a cluster by repeatedly including neighboring points that satisfy density-reachability. This expansion continues until no more points can be added. Border points are assigned to clusters during this expansion, while isolated points remain labeled as noise.

7.7 Computational Complexity

The time complexity of DBSCAN is $O(n \log n)$ when an efficient spatial index (such as a KD-tree or R-tree) is used to query neighborhoods. Without spatial indexing, the worst-case complexity becomes $O(n^2)$, making it unsuitable for very large datasets.

7.8 Parameter Estimation Based on Nearest Neighbour Analysis

A principled way to estimate the DBSCAN parameters ε and MinPts is through nearest neighbour (NN) distance analysis, which directly reflects the local density structure of the data. Since DBSCAN defines clusters based on point neighborhoods, nearest neighbour distances provide a natural and intuitive basis for parameter selection.

7.8.1 Nearest Neighbour Distance Distribution

For each data point x_j , the distance to its k -th nearest neighbour is computed, where $k = \text{MinPts} - 1$. These distances capture the local point density around each data point. In dense regions, the k -NN distances are small, whereas in sparse regions and noise, they are relatively large.

7.8.2 Choosing MinPts Using Nearest Neighbours

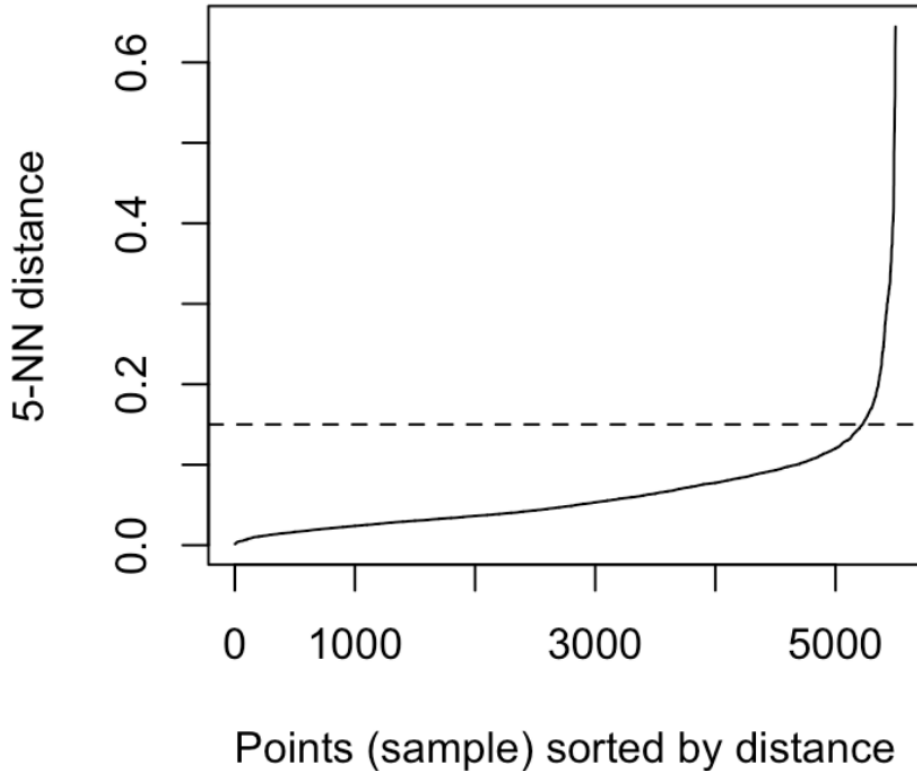
The parameter MinPts controls the minimum local neighborhood density required to form a cluster. From a nearest neighbour perspective, MinPts determines the order of the neighbour whose distance is analyzed. A commonly used heuristic is:

$$\text{MinPts} \geq d + 1$$

where d is the data dimensionality. Increasing MinPts corresponds to considering higher-order nearest neighbours, which enforces stricter density requirements and improves robustness to noise.

7.8.3 Estimating ε via k -Nearest Neighbour Distances

After fixing MinPts, the distance to the k -th nearest neighbour is computed for every data point. These distances are then sorted in ascending order to obtain the k -distance curve. The optimal value of ε is selected at the point where this curve shows a sharp change in slope (the elbow or knee point). This transition indicates a boundary between dense clusters and sparse noise regions. See the figure below for illustration.



7.8.4 Interpretation in Terms of Density

From a density viewpoint, ε defines the radius within which at least MinPts neighbours must exist for a point to be considered a core point. Nearest neighbour analysis ensures that this radius adapts to the intrinsic spacing of points in dense regions rather than relying on arbitrary distance thresholds.

7.8.5 Practical Considerations

Nearest neighbour-based estimation is sensitive to feature scaling; hence, data normalization is essential before computing NN distances. In datasets with highly varying densities, a single global ε derived from nearest neighbour analysis may be insufficient, motivating the use of adaptive density-based methods such as OPTICS or HDBSCAN.

7.9 Advantages

- Discovers clusters of arbitrary shape and size
- Does not require the number of clusters to be specified in advance
- Naturally identifies noise and outliers
- Robust to small variations in data

7.10 Limitations

- Highly sensitive to the choice of ε and MinPts
- Struggles with datasets containing clusters of varying densities
- Performance degrades in high-dimensional spaces due to distance concentration

7.11 Applications

- Spatial data analysis and geographic information systems (GIS)
- Anomaly and fraud detection
- Image segmentation and pattern recognition
- Network intrusion detection